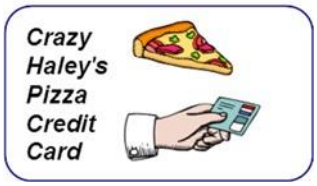
Lab Week 12 – Assignment 4 – Problem Details



Congratulations! You have been contracted by Crazy Haley's Pizza to write a software application that automates the current manual process used to validate a Crazy Haley's Pizza Credit Card. These are unique credit cards used by *"thousands and thousands"* of students whose use their laptops in class to order pizza for everyone after their cell phones go off in class! The uniqueness of these credit cards is that the card numbers are 4 digits in length.

The Current Manual Process of Credit Card Validation



Once the delivery person has received a Crazy Haley's Pizza Credit Card from the student, a manual check of the card number is conducted using a pencil and paper to perform all the calculations required to validate it.

An example of the calculations required follows, based upon the Luhn algorithm.

Crazy Haley's Pizza Credit Card (CHPCC) Validation Algorithm

The Credit Card Number is written down by the delivery person

1	2	3	4	<-- **Digit Num**ber
7	7	3	2	

Next, the validity of the Credit Card Number is checked using the following procedure:

Double the values of the odd-numbered digits of the Credit Card Number,

7	**7**	**3**	**2**
x2		**x2**	
14		6	

Cross-add the multiplied values > 9

(Note that 14 yields the separate values of 1 and 4, hence 1 + 4 = 5)

Thus the sum of the products is:

5	**+**	**6**		**=**	**11**

The sum of the even-numbered digits of the Credit Care Number is:

	7	**+**	**2**	**=**	**9**
				Total	**20**

The total found by following the preceding steps
must be evenly divisible by 10 (Remainder must be 0) for it to be a valid Credit Card Number

Analysis of Current Card Number **Valid Card Number**

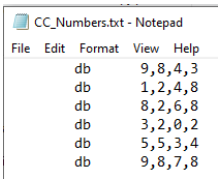
The Future Automated Process of Credit Card Validation

In the business of Software Engineering many manual processes are automated to improve efficiency, risk management and customer satisfaction.

MagTek MiniUSB Stripe Reader Card Reader



In a fully implemented system, we would swipe our credit cards using a device such as the one depicted to the right. However, before adding such a device to our system, we would first have to prove that the functionality of the software is correct. As such, we will simulate the actual reading of the credit card by the use of an external data file that contains exactly six credit card numbers, which we will read into our assembly language program.



We will validate each of six the credit card numbers, keeping track of the Number of Valid Credit Cards and the Number of Invalid Credit Cards as we iterate through the values. An example result is illustrated below.

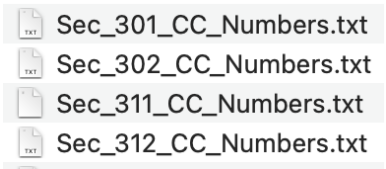
Development Requirements and Constraints

1. You must use the supplied skeleton code for ***CC_Validation.asm*** as your starting point. As you will see, some of the code has been predefined for your mandatory use and annotated "Do NOT change" within that file. Pay particular attention to the label for your "results", ensuring that the two memory locations allocated for the program's results are utilized; otherwise, the supplied code that outputs the results to the HEX displays will not correctly function.

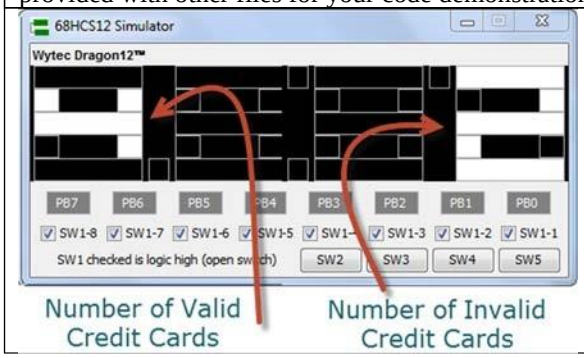
2. Program "stubs" are also supplied for the three subroutines that you must write – AddEven.asm, AddOdd.asm and Validate_CC.asm, where a program "stub" is basically a blank code listing that you complete **using the supplied subroutine template**.

3. The first thing you should do is to create a test plan of your program run **BEFORE ANY CODE IS WRITTEN**. It must contain the calculations that prove the validity of your results using the CHPPC Validation Algorithm. Your test plan must be based upon the file of Credit Card numbers that you will "include" in your "main" program as follows for your course section:





Only ***after*** we determine the status of ***all*** of the six credit cards, will we will display the results on the 7-Segment Displays of the Simulator. Here is an **example** program run for the illustrated credit card number file, which will be provided to you to test your solution. You will also be provided with other files for your code demonstration.



Note that "CC_Numbers.txt" has also been supplied. It contains the Credit Card numbers used in the example program run. I would encourage you to start off using that file as the results have been supplied to you as well as a partial example Test Plan that will walk you through WHAT task of each subroutines must perform in this assignment. You should complete the missing entries in that document so that you can confirm the functionality of your software. Then, once you understand what problem you are trying to solve and eventually code it, you know that the answer should be **4** and **2** as depicted above. Once you have that working, you can then use your own file of Credit Card numbers to validate your solution AFTER you have completed its Test Plan

Note that I will ask to see your test plan (and other rough work) should you run into difficulty with this assignment, as the test plan should reveal to you the inner workings of each of the subroutines.

Coding

4. Write a "main" Assembly Language program called ***CC_Validation.asm*** that determines the Number of Valid Credit Cards and the Number of Invalid Credit Cards as we iterate through the values. Use the following high-level program flow, in the order given, when writing ***CC_Validation.asm***, noting that there will be other things that "main" must also perform as it uses the named subprograms.

   a. Initialize Program Constants and Variables;
   b. do
      i. jsr Add_Odd
      ii. jsr Add_Even
      iii. jsr Validate_CC
      iv. store results of Validate_CC
   until all six Credit Cards are validated

   c. jsr Config_HEX_Displays
   d. do
      i. jsr Hex_Display
      ii. jsr Delay_ms
   forever

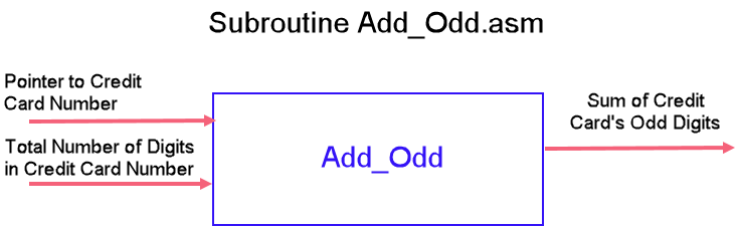
   This code has been supplied to you!

5. In support of ***CC_Validation.asm***, you must have "main" use three subroutines that manipulate the credit card digits and validate them based upon the example use of the Luhn algorithm illustrated in this document and the sample Test Plan provided to you. I have also provided you with a subroutine template.

6. As such, you must write the following Assembly Language subroutines and place them in your Source Code folder.

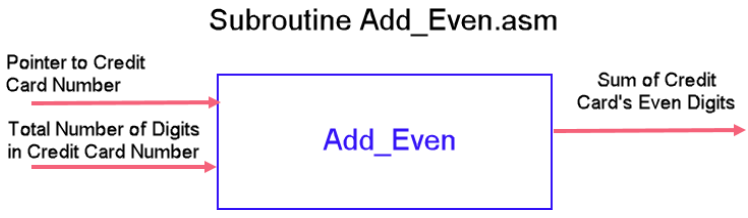
Add_Odd

a. this subroutine is called by "main" using **jsr Add_Odd**
b. Its purpose is to add up the odd numbered digits (digits 1 and 3), doubling each digit and cross-adding as required.
c. You MUST use **use iteration in your solution** so that the subroutine can be used with a credit card with any Total Number of Digits
d. Its filename is ***Add_Odd.asm*** *and it is to be stored in your source folder.*



Add_Even

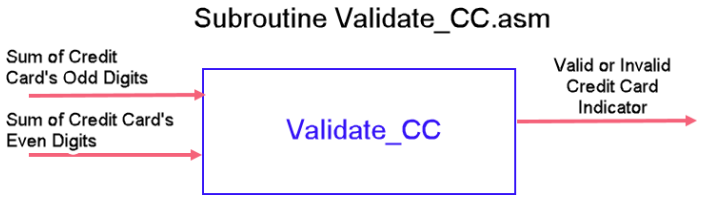
a. This subroutine is called by "main" using **jsr Add_Even**
b. Its purpose is to Add up the even numbered digits (digits 2 and 4)
c. You MUST use **use iteration in your solution** so that the subroutine can be used with a credit card with any Total Number of Digits
d. Its filename is ***Add_Even.asm*** *and it is to be stored in your source folder.*



Validate

a. this subroutine is called by "main" using **jsr Validate_CC**

b. Its purpose is to add the Sum of Credit Card's Odd and Even Digits and determine if the sum is evenly divisible by 10 (e.g. a Remainder of 0 when divided by 10). If it is, then the Credit Card number is valid; otherwise, it is not valid. **This** subroutine makes this decision, **not** "main"



c. Note that iteration is **not** to be used in the subroutine because it can only validate one Credit Card at a time.

d. Its filename is ***Validate_CC.asm*** *and it is to be stored in your source folder.*

Good Programming Practices Regarding Subroutines created in separate files

i. Each subroutine is to be in a separate file and fully documented as per the documentation standards contained in the provided subroutine template. Further examples of these standards can be viewed in ***Config_Hex_Displays.asm*** *and* ***Hex_Display.asm*** *which are supplied to you for this lab.*

ii. Subroutines are independent bodies of code and should have "no knowledge of each other." [e.g. In C, toupper() doesn't know about tolower()].

iii. Subroutines, including library files, may only be called by "main."

iv. Subroutines must not contain an **org** statement.

v. While Subroutines have access to the global values contained in ***registers.inc***, subroutines cannot use "global" constants or other values defined by **equ** statements in "main." Rather, subroutines may contain their own **equ** statements for any values that are local to them.

vi. Subroutines are called by a **jsr** to the label that names the subroutine (look at ***Hex_Display.asm*** for an example).

vii. Subroutines must be "passed" pointers (pass by reference) , and other values in registers (pass by value) from "main." These values must be documented in the Pre-Condition section of the subroutine header.

viii. Subroutines cannot modify memory in "main" or other subroutines' memory space. Rather, they must "return" values in registers. These values must be documented in the Post-Condition section of the subroutine header. Do not use the stack to return values to main.

ix. Within a subroutine, where any registers' contents are destroyed (overwritten with new values), this must be documented in the Post-Condition section of the subroutine header.

x. A subroutine may have one or more **rts** statements

xi. A subroutine should not be independently assembled by itself as you may received unexplanable errors if you do so; however, a subroutine should be "saved" after making any changes its code.

xii. Since subroutines will "destroy" register contents, "main" must protect any register content that will be used after the subroutine execution. For example, if Accumulator A is used as a counter in main(), then Accumulator A must be protected in main(), as it is not the subroutine's task to protect values from main.

Approach to Development

There are at least a couple of methods one can use in Software Development:

- the "Big Bang" approach where all of the code is written at once and then "fingers are crossed" hoping that the code works first time; or
- the "Incremental" approach where we "code a bit, test a bit." That is, we start off with our basic "skeleton code" in main(), then add functionality to the program as we go along. In this assignment, you may wish one group member to code **Add_Odd.asm** while another one codes **Add_Even.asm**. You can test these individual subroutines using mini drivers that send a single Credit Card Number to the subroutine and see if you get the predicted results. Once you have that functionality working you can incorporate it into main(), retest it and then start looking at the iteration portion of main() where it must loop through all of the credit cards, updated the Valid and Invalid Results as appropriate. The same approach could be taken for **Validate_CC.asm**, which should be the easiest subroutine to write for this assignment

Do not hesitate to ask for assistance if you need it for this assignment; however, be prepared with a completed Test Plan, perhaps a Flow Chart and other rough work so that we can see WHAT problem you are attempting to solve.

HCS12 Instructions you are likely to use in this Lab Assignment

S12CPUV2 Reference Manual, Rev. 4.0

Table 5-1. Load and Store Instructions

Mnemonic	Function	Operation
	Load Instructions	
LDAA	Load A	(M) ⇒ A
LDAB	Load B	(M) ⇒ B
LDD	Load D	(M : M + 1) ⇒ (A:B)
LDS	Load SP	(M : M + 1) ⇒ SP_H:SP_L
LDX	Load index register X	(M : M + 1) ⇒ X_H:X_L
LDY	Load index register Y	(M : M + 1) ⇒ Y_H:Y_L

Table 5-2. Transfer and Exchange Instructions

Mnemonic	Function	Operation
	Transfer Instructions	
TAB	Transfer A to B	(A) ⇒ B
TBA	Transfer B to A	(B) ⇒ A
TFR	Transfer register to register	(A, B, CCR, D, X, Y, or SP) ⇒ A, B, CCR, D, X, Y, or SP

Table 5-6. Decrement and Increment Instructions

Mnemonic	Function	Operation
	Decrement Instructions	
DEC	Decrement memory	(M) – $01 ⇒ M
DECA	Decrement A	(A) – $01 ⇒ A
DECB	Decrement B	(B) – $01 ⇒ B
	Increment Instructions	
INC	Increment memory	(M) + $01 ⇒ M
INCA	Increment A	(A) + $01 ⇒ A
INCB	Increment B	(B) + $01 ⇒ B
INX	Increment X	(X) + $0001 ⇒ X
INY	Increment Y	(Y) + $0001 ⇒ Y

Table 5-9. Clear, Complement, and Negate Instructions

Mnemonic	Function	Operation
CLR	Clear memory	$00 ⇒ M
CLRA	Clear A	$00 ⇒ A
CLRB	Clear B	$00 ⇒ B

Table 5-10. Multiplication and Division Instructions

Mnemonic	Function	Operation
	Division Instructions	
IDIV	16 by 16 integer divide (unsigned)	(D) ÷ (X) ⇒ X Remainder ⇒ D

Table 5-12. Shift and Rotate Instructions

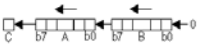
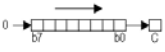
Mnemonic	Function	Operation
	Logical Shifts	
LSL LSLA LSLB	Logic shift left memory Logic shift left A Logic shift left B	
LSLD	Logic shift left D	
LSR LSRA LSRB	Logic shift right memory Logic shift right A Logic shift right B	

Table 5-21. Jump and Subroutine Instructions

Mnemonic	Function	Operation
BSR	Branch to subroutine	SP – 2 ⇒ SP RTN_H : RTN_L ⇒ $M_{(SP)}$: $M_{(SP+1)}$
JSR	Jump to subroutine	SP – 2 ⇒ SP RTN_H : RTN_L ⇒ $M_{(SP)}$: $M_{(SP+1)}$ Subroutine address ⇒ PC
RTS	Return from subroutine	$M_{(SP)}$: $M_{(SP+1)}$ ⇒ PC_H : PC_L SP + 2 ⇒ SP

Table 5-23. Index Manipulation Instructions

Mnemonic	Function	Operation
	Addition Instructions	
ABX	Add B to X	(B) + (X) ⇒ X
ABY	Add B to Y	(B) + (Y) ⇒ Y
	Compare Instructions	
CPX	Compare X to memory	(X) – (M : M + 1)
CPY	Compare Y to memory	(Y) – (M : M + 1)

Table 5-24. Stacking Instructions

Mnemonic	Function	Operation
	Stack Operation Instructions	
PSHA	Push A	(SP) – 1 ⇒ SP; (A) ⇒ $M_{(SP)}$
PSHB	Push B	(SP) – 1 ⇒ SP; (B) ⇒ $M_{(SP)}$
PSHX	Push X	(SP) – 2 ⇒ SP; (X) ⇒ $M_{(SP)}$: $M_{(SP+1)}$
PSHY	Push Y	(SP) – 2 ⇒ SP; (Y) ⇒ $M_{(SP)}$: $M_{(SP+1)}$
PULA	Pull A	($M_{(SP)}$) ⇒ A; (SP) + 1 ⇒ SP
PULB	Pull B	($M_{(SP)}$) ⇒ B; (SP) + 1 ⇒ SP

Your thought-process, coupled with flowcharts and your test plan should lead you to the various instructions that are typically used to code this assignment. For example, the given algorithm states that you must divide a certain result to see if the answer is evenly divisible by 10 – e.g. that the remainder is 0. Given that we are working with unsigned integer values, which should lead us to the ***idiv*** instruction. To see how this, or any instruction works, make up a small test program, set up the registers appropriately, execute the instruction and observe the results – e.g. make wise use of the simulator's debugging capabilities.

The same holds true for any code you develop – test it in the simulator – if you do not get the expected results, single step the code in the simulator to discover where the problem exists.

As discussed in class, if you have worked out WHAT you want to do, but are stuck for an instruction to perform the task (given that you have already looked for an instruction), I would be happy to point you in the right direction. HOWEVER, I can only do that if you have a Test Plan, perhaps a Flowchart and other rough work.

I cannot help with code, however, when I do not know what problem you are attempting to solve because we all have different ways to solve a problem – e.g. Be at the Rideau Centre at 5:00 p.m. today – *I could take the bus, you could take a taxi, another student might walk there, and so on…* We all solved the problem, but in different manners.